Computer Engineering Technology Department
Software Engineering Program

SET 2430 – Algorithmic Design and Data Structures

COURSE CREDIT: 3
2 lecture hours, 2 laboratory hours

Fall 2026

Day: Wednesday & Thursday	**Time:** **Lecture: Wednesday** 9:00AM - 10:40AM **Lab: Thursday** 9:00AM - 10:40AM	**Location:** **Wednesday:** Online-Zoom **Thursday**: V-610/online

INSTRUCTOR

Professor: Haiying Xiao	
Email: *haiying.xiao30@citytech.cuny.edu*	**Office Hours:** Tuesday, 11:45AM–12:45 PM **Room:** V-631

COURSE DESCRIPTION

This course introduces core data structures and algorithms essential for efficient software development. Students will implement and analyze common data structures and algorithmic techniques while solving practical programming problems. Topics include linear and non-linear data structures, searching and sorting algorithms, recursion, hash tables, graphs, and algorithm efficiency analysis using Big-O notation.

TRAINING QUALIFICATIONS

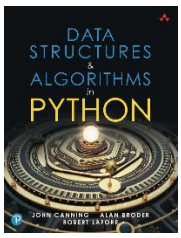
SET 2430 course focuses on the fundamental concepts and techniques required to design, analyze, and implement efficient data structures and algorithms. Students learn how data is organized, stored, and manipulated to solve computational problems effectively. The course introduces core data structures and algorithmic strategies through problem-solving, code implementation, and performance analysis. By the end of the course, students demonstrate their ability to select appropriate data structures and algorithms, implement them in code, and evaluate their efficiency using standard analytical methods.

By the end of this course, students will be able to:

- Explain the purpose and characteristics of fundamental data structures, including arrays, linked lists, stacks, queues, trees, hash tables, and graphs.
- Implement common data structures and algorithms using a programming language to solve practical programming problems.
- Analyze algorithm efficiency using time and space complexity concepts, including Big-O notation.
- Select appropriate data structures and algorithms based on problem requirements and performance constraints.
- Apply algorithmic techniques such as recursion, searching, sorting, and traversal.
- Debug, test, and optimize algorithm implementations to improve correctness and efficiency.
- Decompose complex problems into manageable components and develop logical, step-by-step solutions.
- Communicate algorithmic ideas clearly and demonstrate critical thinking, persistence, and initiative while working independently or collaboratively.

COURSE MATERIALS

RECOMMENDED TEXTBOOK



Data Structures & Algorithms in Python by John Canning, Alan Broder, Rober Lafore

ISBN-13: 978-0-13-485568-4

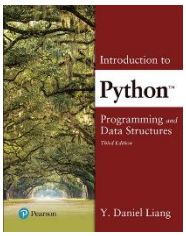
ISBN-10: 0-13-485568-X

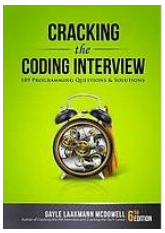
Note: It is not mandatory to have the textbook

Ref. Text Books:

Introduction to Python Programming and Data Structures, 3e

Y.Daniel Liang





Cracking the coding interview: 189 programming questions and solutions.
McDowell, G. L. (2021). Palo Alto, CA:CareerCup, LLC.

GENERAL INFORMATION

ATTENDANCE: 1-19 minutes is considered ONE LATE, THREE lateness is equal to ONE ABSENCE, and THREE absences result in a "WF" or "F" as the lab grade. More than 20 minutes late counts as an absence.

LECTURE CONDUCT: No eating, drinking, smoking, radio playing, cellphones, and use of inappropriate words in the classroom.

CLASS ASSIGNMENTS: There are a total of 10 Class Assignments during the semester. Students are required to perform satisfactory work in the lecture and laboratory to receive a passing grade in the course. Each Class Assignment is done individually during lab schedule. All Class Assignments MUST be submitted before the due date, late Class Assignments will not be accepted, and it will be graded as "zero".

PROJECTS and EXAMS: There are a total of 1 main project, 4 quizzes, 1 midterm and 1 final exam during the semester. All ProjectsMUST be typed, presented in a professional manner, and submitted before the due date, late Projects will not be accepted, and it will be graded as "zero".

Academic Integrity policy (department or College):

Students and all others who work with information, ideas, texts, images, music, inventions, and other intellectual property owe their audience and sources accuracy and honesty in using, crediting, and citing sources. As a community of intellectual and professional workers, the College recognizes its responsibility for providing instruction in information literacy and academic integrity, offering models of good practice, and responding vigilantly and appropriately to infractions of academic integrity. Accordingly, academic dishonesty is prohibited in The City University of New York (CUNY) and at New York City College of Technology (City Tech) and is punishable by penalties, including failing grades, suspension, and expulsion. The college's policy on Academic Integrity can be found at Academic Integrity Policy 2025-2026

Disabilities

City Tech is committed to supporting the educational goals of enrolled students with disabilities in the areas of enrollment, academic advisement, tutoring, assistive technologies, and testing accommodations. If you have or think you may have a disability, you may be eligible for reasonable accommodations or academic adjustments as provided under applicable federal, state, and/or city laws. You may also request services for temporary conditions or medical issues under certain circumstances. If you have questions about your eligibility and/or would like to seek accommodation services and/or academic adjustments, please contact the Student Accessibility Center (SAC) at 300 Jay Street. Room L-237; telephone: 718-260-5143; WWW: http://www.citytech.cuny.edu/accessibility/.

GRADING POLICY

ITEM	Grade %
Class Assignments (10)	40%
Project	10%
Midterm	15%
Quizzes	10%
Final Exam	20%
Attendance	5%
TOTAL	**100%**

NUMERICAL VALUES		
LETTER	**POINTS RANGE**	**GPA**
A Excellent	96-100	4.0
A-	90-95	3.7
B+	87-89	3.3
B Good	84-86	3.0
B-	80-83	2.7
C+	77-79	2.3
C Satisfactory	74-76	2.0
C-	70-73	1.7
D+	67-69	1.3
D Passing	64-66	1.0
D-	60-63	0.7
F Failure	0-59	0

COURSE OUTLINE

Topics
Week 1 Topic: Introduction & Python Refresher *Week 1: Wednesday (Sep 2), Thursday (Sep 3) in person class* In this session, students will learn the role of data structures and algorithms in software development: • Understand the course scope, expectations, and the importance of data structures and algorithms. • Review Python syntax, variables, control flow, functions, and basic data types. • Set up a development environment and practice simple Python exercises. • Introduce Abstract Data Types (ADTs) and basic data structures, understanding the difference between what operations a structure supports (ADT) and how it is implemented (implementation). **Class Assignment 1**
Week 2 Topic: Python Programming Fundamentals & OOP *Week 2: Wednesday (Sep 9), Thursday (Sep 10)* In the second week, students will: • Review Python syntax and write Python functions, understanding parameters, return values, and basic scope rules. • Explore Python collections: lists, tuples, dictionaries, and sets. • Learn Object-Oriented Programming (OOP) in Python: classes and objects, attributes and methods, constructors (__init__), Basic inheritance • Practice reading, writing, and debugging Python code, including simple OOP examples • Emphasize clean syntax, readability, and Pythonic conventions. • *Compare Python syntax with Java, including: Indentation vs. braces for code blocks; Dynamic vs. static typing; Lists vs. arrays; Dictionaries vs. Java Map* **Class Assignment 2**
Week 3 Topic: Mathematical & Algorithm Foundations *Week 3: Wednesday (Sep 16), Thursday (Sep 17) in person class* Learning Objectives: • Introduce essential mathematical tools for algorithm analysis, including sets, relations, common mathematical notations, summations, and logarithms. • Understand recurrence relations and how they describe the performance of recursive algorithms. • Learn basic proof techniques for validating algorithm correctness. • Gain an initial understanding of estimation and approximation for algorithm efficiency. **Class Assignment 3**
Week 4 Topic: Algorithm Analysis (Part 1) *Week 4: Wednesday (Sep 23), Thursday (Sep 24)* • Learning Objectives: • Understand the relationship between problems, algorithms, and programs. • Compare algorithms and analyze their best, worst, and average-case performance. • Learn asymptotic analysis (Big-O notation) to evaluate algorithm efficiency. • Discuss the impact of algorithm efficiency versus hardware performance on program execution. **Class Assignment 4**

Week 5 Topic: Algorithm Analysis (Part 2)
Week 5: Wednesday (Sep 30), Thursday (Oct 1)

- Learning Objectives:
- Understand lower bounds and Θ (Theta) notation for analyzing algorithm performance.
- Calculate program running time and analyze algorithms with multiple input parameters.
- Introduce space complexity and strategies for efficient code design and tuning.
- Practice empirical analysis of algorithms using examples to measure performance.
- Apply problem-solving skills to evaluate algorithm efficiency and optimize implementations.

Class Assignment 5

Week 6 Topic: Linear Structures: Arrays, Lists
Week 6: Wednesday (Oct 7), Thursday (Oct 8)

Learning Objectives:

- Understand the List Abstract Data Type (ADT) and its core operations.
- Implement **array-based lists** and **linked lists** in Python.
- Analyze and compare **performance trade-offs** between arrays, linked lists, stacks, and queues.
- Solve practical problems using **linear data structures**.

Class Assignment 6

Week 7 Topic: Linear Structures: Stacks and Queues
Week 7: Wednesday (Oct 14), Thursday (Oct 15) in person class

Learning Objectives:

- Understand **stack and queue ADTs** and implement them using **arrays or linked lists**.
- Apply stacks and queues to practical problems, such as expression evaluation and task scheduling.
- Explore priority queues and understand their use cases in real-world applications.
- Analyze and compare **performance trade-offs** for stacks, queues, and priority queues

Project

Week 8 Topic: Review & Midterm
Week 8: Wednesday (Oct 21), Thursday (Oct 22) in person class

Review key concepts from Weeks 1–7, including:

- Algorithm analysis (Big-O, performance trade-offs)
- Linear structures (arrays, lists, stacks, queues, priority queues)
- Recursion basics
- Review class assignments and group project, reinforcing practical applications of the topics.

Midterm

Week 9 Topic : Design Patterns & Dictionaries
Week 9: Wednesday (Oct 28), Thursday (Oct 29)

Learning Objectives:

- Understand **design patterns** for data structures and their application in Python.
- Compare **alternative list designs** and explore trade-offs in different implementations.
- Learn the **Dictionary Abstract Data Type (ADT)** and its core operations.
- Implement **dictionary-like structures** using Python and apply them to practical problems.

Week 10 Topics: Binary Tree Basics (Part 1)

Week 10: Wednesday (Nov 4), Thursday (Nov 5)

- Introduction to binary trees as a recursive data structure.
- Review basic tree traversal algorithms: preorder, inorder, postorder.
- Understand recursive information flow and space requirements in tree structures.

Class Assignment 7

Week 11 Topics: Binary Trees (Part 2)

Week 11: Wednesday (Nov 11th), Thursday (Nov 12th)

Learning Objectives:

- Review of Binary Tree Basics
- Implement binary search trees (BSTs) and basic operations: insert, search, delete.
- Apply BSTs for dictionary-like storage and retrieval.
- Explore composite-based expression trees in practical applications.
- Discuss practical considerations in tree-based data structures, including space and efficiency.

Class Assignment 8

Week 12 Topic: Sorting Algorithms (Part 1: Fundamentals)

Week 12: Wednesday (Nov 18), Thursday (Nov 19)

Learning Objectives:

- Introduce sorting terminology and notation (stability, in-place, comparisons).
- Analyze the cost of sorting algorithms using time complexity.
- Implement insertion sort and exchange-based sorting in Python.
- Understand why simple sorting algorithms perform well on small or nearly sorted datasets.

Class Assignment 9

Week 13

Week 13: Wednesday (Nov 25), Thursday (Nov 26) Thanks giving break – no class

Students self-study the topics covered in Weeks 14 & 15.

Week 14 Topic: Sorting Algorithms (Part 2: Efficient & Advanced Methods)

Week 14: Wednesday (Dec 2), Thursday (Dec 3)

Learning Objectives:

- Understand divide-and-conquer strategies in sorting.
- Implement merge sort and quick sort and analyze their performance trade-offs.
- Introduce radix sort as a non-comparison-based sorting algorithm.
- Conduct empirical performance comparisons of sorting algorithms using Python.
- Select appropriate sorting methods for practical data problems.

Class Assignment 10

Week 15: Topic: Hashing

Week 15: Wednesday (Dec 9), Thursday (Dec 10) in person class

Learning Objectives:

- Introduce hashing concepts and the motivation for using hash tables.
- Understand hash function principles and characteristics of good hash functions.
- Implement hash tables in Python using basic hash functions.
- Handle collisions using common strategies (e.g., chaining, open addressing).
- Apply hashing for efficient data storage, lookup, and retrieval.
- Analyze the average-case performance of hashing operations.

Graphs

- Introduce graph terminology and common graph representations (adjacency list, adjacency matrix).
- Implement graph data structures in Python.
- Implement and apply graph traversal algorithms: Depth-First Search (DFS) and Breadth-First Search (BFS).
- Understand and apply topological sorting for dependency-based problems.
- Introduce shortest path and minimum spanning tree problems at a conceptual and implementation level.
- Analyze the time and space complexity of common graph algorithms.

In class Activities

Week 16 Topic: Review & Final Exam

Week 17: Wednesday (Dec 16), Thursday (Dec 17) in person class

- Comprehensive review of all course topics.
- Address student questions and clarify key concepts.

Final Exam

NOTE: Instructor reserves the right to modify the present outline anytime